

https://github.com/code-heim/go_21_goroutines_pipeline.
https://www.codeheim.io/ - Given the requirements in Assignment 6: Data Pipelines with Concurrency. We will continue with our Spec-First, Evidence-Driven Development approach using Claude as the primary builder AI (Claude Code) partner and Gemini as the reviewer AI. We will use the same approach as in Week 5 and prior assignments. Following Spec-First, Evidence-Driven Development (SFEDD) we will be using github as before.

Checking the project notes before continuing Week 6.
I'll pick up the SFEDD workflow from Week 5 and start by pulling the context from prior weeks plus the original CODEHEIM repo.

Read memory, read a page, and 13 more steps ▸
Confirmed: the input file was replaced by a 500×500 grayscale copy. Now the timing question that decides the design: how much can a one-goroutine-per-stage pipeline gain?

Created 5 files, edited a file, and 12 more steps ▸
The Week 6 spec phase is done, in the same shape as Week 5's first commit: the constitution, the assignment summary, Spike 001 and draft Spec 001. They're committed locally on top of CODEHEIM's three original commits, so the history shows exactly what we changed.

What Spike 001 found (I built and ran the original unchanged; it works in 0.35 s):

- **The original design can't go much past ×2, however many cores the Mac has.** With one goroutine per stage, speed is set by the slowest stage. Resize takes 49% of the work, so the ceiling is about ×2.05.
  - Measured: ×1.45–1.47 for 4, 16 and 64 images.
  - A worker pool, where each goroutine handles a whole image, already did better (×1.63–1.73) on only 2 CPUs.
  - This is the main point for management: how the work is divided between goroutines matters as much as using them.
- **Every bad file panics and takes all the images down with it:** a missing file, a corrupt file, a PNG, or a missing output folder.
- **Data-loss bug:** if an input path doesn't contain `images/`, the program overwrites the original photo with its grayscale copy. I confirmed this with a checksum.
- **Two things to know when you pick your photos:**

    - Go ignores the rotation tag in phone photos, so some may come out sideways.
    - The fixed 500×500 resize stretches anything that isn't square.

Your decisions (spec section 11, Q1–Q10). Each has a recommendation; the ones that matter most are:

- **Q1 – repository:** `mioluigi/go-image-pipeline`, keeping CODEHEIM's history and license.
- **Q2 – images:** four of your own photos, at least two of them not square.
- **Q3 – modes:** sequential, the original pipeline, and a worker pool. The worker pool is the first thing to cut if time runs short.
- **Q4 – aspect ratio:** do the optional item; the longer side becomes 500.

Next steps:

1. Send the four documents and the review prompt to Gemini.
2. Reply here with Gemini's findings and your Q1–Q10 answers. I'll record the review and produce spec v2, then the plan and task list.
3. Create the empty GitHub repo (no README or license). Creating a public repo is your call, so I won't do it myself. Once it exists, I'll attach it and push.

The Canvas page shows 11/1/2026, which I read as due tonight. That's why the spec's Q3 marks the worker pool as the first cut and asks Gemini what else could go.

Download all



Created a file, ran 3 commands ▸

Spec v2 is done. Given tonight's deadline, I'll also draft the plan and task list now, so you can approve all three in one pass instead of three round trips.

Created 2 files, shared 4 files, ran a command ▸

I've triaged Gemini's review: 3 findings accepted, 3 accepted with changes, 1 rejected. Spec v2 applies the changes. Because of tonight's deadline, I also drafted the plan and the task list so you can approve everything in one pass.

How the review changed the spec:

- **Concurrency safety (Gemini's one High finding): requirement accepted, mechanism not.**
  - Gemini wanted the plan to require `context` cancellation. The spec now states what a grader can test instead: after any run, including runs with failed images, the goroutine count returns to where it started.
  - The plan explains why cancellation isn't needed. Errors travel inside each image's job, so no stage ever stops reading early.
  - Thinking this through exposed a second risk Gemini missed: the concurrent modes finish images out of order. The spec now requires results in input order.
- **Small fixes:**
  - The coverage check now names its command (`go test -cover`).
  - Exit codes are defined: 0 = all images saved, 1 = some image failed, 2 = bad usage.
  - A new table maps all ten assignment steps to requirements.
- **One correction to Gemini:** the ≈×2 ceiling doesn't depend on CPU count. It comes from how the work splits across stages, and that split will change with your photos and the new resize. So the spec now says the ceiling gets recomputed from your Mac's own stage timings.
- **Rejected: making the `.app` a stretch goal.** The rubric's Application criterion asks for it, and the Week 5 build script makes it a few minutes' work. Instead the spec now has a cut order if the evening runs short: worker pool first, then the measurement tool.

The review PDF is saved in `docs/ai-dialogs/`. Locally, the repo has three commits of ours on top of CODEHEIM's three.

What I need from you (task T00):

1. Approve or change Q1–Q10. Gemini agreed with all ten recommendations, and the plan assumes them.
2. Approve the plan and the task list, or tell me what to change.

3. Create an empty `mioluigi/go-image-pipeline` on GitHub, with no README, license or .gitignore. I'll then push the history and start coding at T01.
4. Meanwhile, start on T02 (your photos): export four of them from Photos as JPEG, each under 5 MB, with at least one landscape and one portrait. Exporting from Photos fixes the sideways-rotation problem.

Download all

Approved all changes and mioluigi/go-image-pipeline has been created in github

Marking the spec, plan and tasks as approved.
Approved — I'll attach the repo, push the history, then start on T01 and keep going down the list.

Used Add repo, ran 2 commands ▸
One snag: GitHub refused my push to `mioluigi/go-image-pipeline` (403 — the Claude GitHub App can't access this new repo). Most likely the app is installed for **selected repositories** only, and the list still holds just last week's repo. To fix it: GitHub → Settings → Applications → **Claude** → Configure → Repository access → add `go-image-pipeline` → Save. If that page doesn't list it, reconnect GitHub from claude.ai Settings → Connectors.

I'm carrying on in the meantime — the approval commit and T01 onward go into the local repo, and I'll push everything the moment access works. Just tell me when it's done.

Created 14 files, ran 37 commands ▸
Phases 0–4's coding is pushed (T01–T11 and T13). The remaining steps need your Mac, so I'll send you the checklist now and draft the README while you run it.
Your Mac steps — T02 (photos), T12 (measurements), T14 (executables). About 20 minutes; run in Terminal:

bash

```
git clone https://github.com/mioluigi/go-image-pipeline.git && cd go-image-pipeline

# T02 — replace CODEHEIM's photos with four of yours
git rm -q images/image*.jpeg
#   Photos app → select 4 photos (at least 1 landscape, 1 portrait) → File → Export →
#   → JPEG, Size: Large or Full Size → save into the repo's images/ folder
ls -lh images                       # each ≤ 5 MB; .jpeg/.jpg/.png only; no two with t
go test ./... && go run . -mode sequential    # should end "4 saved, 0 failed"
git add images && git commit -m "T02: replace CODEHEIM's images with four of my own p

# T12 — official measurements (plugged in, other apps closed; ~10 minutes)
scripts/measure.sh
scripts/tables.sh > results/tables.md
git add results && git commit -m "T12: measurements on the Mac" && git push

# T14 — executables
scripts/build.sh && open dist/ImagePipeline.app
```

The `.app` opens Terminal and runs all three modes in turn, writing to `~/Desktop/ImagePipeline-output` — glance at one output photo to confirm it's upright and not stretched. If any step stops with an error, paste it here. While you run these, I'm writing the README (T15); its tables fill in automatically from your `results/`.

Created a file, ran a command ▸

The code is done and on GitHub: tasks T01–T11 and T13, plus a README draft. The only steps left are the ones on your Mac, and the commands are in my previous message.

What's built:

- **Tests:** everything passes under the race detector. Coverage is 100% for both reusable packages and 96% for the program.
- **Mutation checks:** of 20 deliberate behaviour-changing bugs, all 20 were caught (wrong result order, channels left open, a goroutine stopping early, inputs overwritten, and others).
- **Correctness of concurrency:** a test confirms all three modes write byte-identical files, and another confirms no goroutine is left running after any run, even when images fail.
- **The original:** `scripts/run-original.sh` builds CODEHEIM's program from its own commit in our history, runs it unchanged, and times it against ours on the same four images.
- **Executables:** the Windows `.exe` and the macOS `.app` both build. The `.app` runs all three modes on the built-in photos so the time comparison shows on screen.
- **README:** drafted with every section the rubric asks for, including the GenAI Tools section. Its tables fill in from `results/` by script.

Two things only you can settle:

- The README's recommendation wording is provisional until your measurements are in. In the 2-CPU workspace trial (not committed), the measured stage times put the pipeline's ceiling at ×1.83, and the worker pool was faster. Your Mac has more cores, so the gap between the two should widen; your numbers will decide the final wording.
- Please check the GenAI Tools statement: Claude Pro as builder (Claude Opus 5.5) and Gemini Ultra as reviewer.

After you push the results (T12), I'll:

1. Generate the tables and finalize the recommendation from your numbers.
2. Check your photos' orientation tags and sizes.
3. Write the Gemini repository-review prompt (T17).

Then you export the AI dialogs (T16), run the final fresh-clone check (T18), and submit `https://github.com/mioluigi/go-image-pipeline.git`.

```
Cloning into 'go-image-pipeline'...
remote: Enumerating objects: 168, done.
remote: Counting objects: 100% (168/168), done.
remote: Compressing objects: 100% (94/94), done.
remote: Total 168 (delta 72), reused 163 (delta 67), pack-reused 0 (from
0)
Receiving objects: 100% (168/168), 2.31 MiB | 16.93 MiB/s, done.
Resolving deltas: 100% (72/72), done.
ls: images: No such file or directory
go: downloading github.com/nfnt/resize v0.0.0-20180221191011-
83c6a9932646
# github.com/mioluigi/go-image-pipeline
images.go:13:12: pattern images: no matching files found
FAIL    github.com/mioluigi/go-image-pipeline [setup failed]
ok      github.com/mioluigi/go-image-pipeline/cmd/measure    0.369s
ok      github.com/mioluigi/go-image-pipeline/imageprocessing    0.319s
ok      github.com/mioluigi/go-image-pipeline/pipeline    0.499s
FAIL
fatal: pathspec 'images' did not match any files
measure.sh: building the program and the measurement tool
images.go:13:12: pattern images: no matching files found
measure.sh: STOPPED because the step above failed; results/ is
incomplete, so do not run tables.sh
tables.sh: missing or empty results/tests.txt (run scripts/measure.sh
first)
[main 92d42c1] T12: measurements on the Mac
 6 files changed, 4 insertions(+)
 delete mode 100644 images/image1.jpeg
 delete mode 100644 images/image2.jpeg
 delete mode 100644 images/image3.jpeg
```

```
 delete mode 100644 images/image4.jpeg
 create mode 100644 results/machine.txt
 create mode 100644 results/tables.md
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 10 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 487 bytes | 487.00 KiB/s, done.
Total 5 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/mioluigi/go-image-pipeline.git
  97b7b7e..92d42c1  main -> main
images.go:13:12: pattern images: no matching files found
```

Show more

Ran 5 commands ▸

Everything's fixed and pushed. Main still won't build until your photos are in, which is the next step anyway.

The cause. Pasted as one block, the commands ran straight past the Photos-export line, which was only a comment. `git rm` emptied `images/`, and the program can't build without at least one photo to embed. So the tests, `measure.sh` and `build.sh` all stopped, and the T12 commit only deleted CODEHEIM's photos and added an empty `tables.md`. Nothing is lost.

Fixed since then. Both scripts now stop with a plain message ("images/ has no .jpg, .jpeg or .png photos; add them first") instead of Go's cryptic error. My first version of that check wrongly rejected a folder of `.JPG` files. I caught that in testing and fixed it; both commits are pushed.

Please run these **one step at a time:**

1. Get the fixes and open the empty photos folder

```bash
cd go-image-pipeline
git pull
mkdir -p images && open images
```

2. By hand, in the Photos app. Select four photos, at least one landscape and one portrait. Then choose File → Export → Export 4 Photos…, set JPEG and Size: Large, and save them into the `images` folder Finder just opened. Check the result:

```bash
ls -lh images        # four photos, each under 5 MB
```

3. Only once step 2 shows your four photos

```bash
go test ./... && go run . -mode sequential      # should end "4 saved, 0 failed"
git add images && git commit -m "T02: four of my own photos" && git push
```

4. Measurements (about 10 minutes, plugged in)

```bash
scripts/measure.sh && scripts/tables.sh > results/tables.md
git add results && git commit -m "T12: measurements on the Mac, with my photos" && gi
```

5. Executables

```bash
scripts/build.sh && open dist/ImagePipeline.app
```

Your machine file already shows an M1 Max with 10 cores. That's a good test of the spec's main prediction: the pipeline should stay below its ceiling of about ×2, while the worker pool should pull clearly ahead. Once step 4 is pushed, I'll check the photos, put the tables in the README, and finalize the recommendation from your numbers.

what is next step

Ran 4 commands ▸

Headline generated. Now the spike addendum, which records that the prediction was wrong for full-size photos, since that's the evidence-driven part of the story:

Ran 2 commands ▸

On second look, "resize writes the same output either way" claims more than I measured: resize also reads every input pixel. I'll state only what the data shows.

Created a file, shared a file, ran 3 commands ▸

The next step is yours: three tasks on your side, then the final check together.

I checked your photos (upright, under 5 MB) and your Mac results, then finished the README (T15). All its numbers are generated from `results/`, including a new headline summary, and it's pushed. Your results showed that on your 12 MP photos, JPEG decoding is the slowest stage, not resize as Spike 001 predicted; I added that to the spike as an addendum. The worker pool came out ×3.81 faster than sequential.

Your next steps, in order:

1. **T14, if you haven't yet:** run `git pull`, then `scripts/build.sh && open dist/ImagePipeline.app`. Check that the Terminal window shows all three modes, and that one photo in `~/Desktop/ImagePipeline-output` looks right: grayscale, upright, and not stretched (500×375).
2. **T17, Gemini review:** paste `gemini-repo-review-002-prompt.md` into Gemini with the repo URL. Bring its findings back here and I'll triage them into `docs/reviews/002-repository-review.md`, as in Week 5.
3. **T16, AI dialogs:** export this Claude conversation and the Gemini repository review as PDFs. Commit them to `docs/ai-dialogs/` (the spec review PDF is already there). Do this last, so the Claude export includes the end of our work.
4. **T18, together:** I run every acceptance check from a fresh clone, then you submit `https://github.com/mioluigi/go-image-pipeline.git` in Canvas.

Two things to confirm when you reply:

- **All-landscape photos.** Decision Q2 asked for at least one portrait. I've recorded that you're keeping these four, unless you'd rather swap one and re-run T12 (about 10 minutes).
- **The README's GenAI Tools section.** It says Claude Pro, model Claude Opus 5.5, as builder, and Gemini Ultra as reviewer.

Gemini repo review 002 prompt
Document · MD

Let's go back to T12. I changed the photos. Let's redo it and complete all the remaining tasks.

Pull and check for the new photos and results ▸

Your new photos aren't on GitHub yet; the latest commit is still my T15. They're only on your Mac, and your copy is also behind mine, so start with a `git pull`. Here are the steps.
Your new photos are only on your Mac so far (GitHub still has the four landscape ones), and your copy is a few commits behind mine. Run these **one block at a time** in `go-image-pipeline`:

1. Catch up without losing your new photos

bash

```
rm -f images/.DS_Store     # macOS file I removed from the repo; it would block the p
git pull
ls -lh images              # your four new photos, each under 5 MB
```

2. Commit the photos (T02 redo)

bash

```
go test ./... && go run . -mode sequential      # should end "4 saved, 0 failed"
git add -A images && git commit -m "T02 redo: new photo set (landscape and portrait)"
```

3. **Re-measure (T12 redo, ~10 min, plugged in, other apps closed)** — this time it also refreshes the README tables:

bash

```
scripts/measure.sh && scripts/tables.sh > results/tables.md && scripts/readme.sh
git add results README.md && git commit -m "T12 redo: measurements on the Mac with th
```

4. **Executables (T14)**

bash

```
scripts/build.sh && open dist/ImagePipeline.app
```

I'm watching GitHub: as soon as step 3 lands I'll check the photos, re-check every README sentence against the new numbers (the slowest stage may change with the new photos), and record the redo. While measure.sh runs, you can do T17 — paste `gemini-repo-review-002-prompt.md` into Gemini — and send me its findings.

Created a file, ran 2 commands, ran terminal ▸

The acceptance check is ready, and on today's state from a fresh clone it passes 15 of 18 checks. The three failures are exactly what's still outstanding:

- **Photos:** a portrait. Your new set should fix this.
- **AI dialogs:** the Claude conversation and Gemini repository-review PDFs (T16).
- **Task list:** the remaining ticks, which I add as each task closes.

I'm watching GitHub for your pushes. When the T12 redo lands, I'll:

- check the new photos;
- go through every README sentence against the new numbers. The slowest stage may change with portrait photos, and the recommendation must follow the data;
- record the redo in the decision log and spike addendum.

Then I'll triage Gemini's findings when you send them and fix what's accepted, and run the final acceptance check. After that, you export the dialogs (T16) and submit.

Your steps are still the four blocks in my last message, starting with `rm -f images/.DS_Store && git pull`.

Your new photos just arrived on GitHub. Checking them while T12 runs on your Mac:

Ran 3 commands ▸

Please stop `scripts/measure.sh` **now (Ctrl-C) if it's running** — two problems with the new photos, one of them a privacy issue:

1. **The "portrait" photo will come out sideways.** `FE3994CD-…JPG` is stored as landscape pixels (3664×2062) with an iPhone "rotate 90°" tag; Go ignores that tag (the limitation in Spike 001), so the program would make a sideways 500×281 image and the set would still have no portrait. It wasn't exported through Photos (its name suggests AirDrop/Files), which is why the rotation wasn't applied. Q2 already approved "re-saved upright if needed," so I'm doing that now.
2. **Three of the four photos contain your GPS location** (IMG_2774, 2846, 3015 — iPhone location data in the file), and the repo is public. I'm stripping all camera metadata (location, date, device) from the four files in the same commit; the pixels of the three landscape photos stay byte-for-byte identical, so nothing else changes.

One decision for you: the *earlier* photo set (IMG_2462/2553/2658/2659, all with GPS) and these three are still in the repo's **history**, so anyone can retrieve them from older commits even after my fix. Options:

- **(a) Rewrite history** to drop the photos from old commits, then force-push. Cleanest; I'd do it carefully, and you'd re-clone afterwards. ← I recommend this.
- **(b) Leave history as is** (locations from 2022 photos stay retrievable).
- **(c) Make the repo private** and add the grader as a collaborator.

Reply **a**, **b** or **c**. Then, after `git pull`, re-run step 3 (measure.sh + tables + readme) — I'll tell you the moment the fixed photos are pushed.

Created a file, read a file, ran 3 commands ▸



The portrait comes out 281×500, upright and not stretched. Committing with the decision recorded. I'm also removing the README's "all landscape" limitation, since it no longer applies.

Commit and push the photo fix ▸

Fixed photos are pushed (`3dd3bbd`). The portrait is now `IMG_portrait.jpg`, stored upright (it comes out 281×500, checked visually), and all four files have no location or other camera metadata. The three landscape photos' pixels are verified identical.

What to do next depends on your a/b/c answer about the photo history:

- **If (a), rewriting history:** reply "a" and **don't run anything yet.** I'll rewrite and force-push (a few minutes), then you re-clone fresh and run step 3 in the new clone.
- **If (b) or (c):** go ahead now: `git pull`, then step 3 (`scripts/measure.sh && scripts/tables.sh > results/tables.md && scripts/readme.sh`, then commit and push).

Ran 6 commands ▸

The dry run is ready and verified:

- **Everything a push would publish (** `main` **) is clean:** 17 photo versions across all commits, none with location data.

- **CODEHEIM's three commits are untouched,** with the same hashes and signatures, so `run-original.sh` still works.
- **The final files are identical to what's on GitHub now;** only the history changes.

The 7 GPS hits were the old remote-tracking copy of GitHub's current state, which a force-push replaces.

One caveat with option (a): GitHub keeps orphaned commits reachable by their exact ID for a while after a force-push. In practice nobody can find them without those IDs, but guaranteed deletion takes a request to GitHub Support. Option (a) still removes the location data from everything the repo shows and from every fresh clone.

Waiting on your reply:

- **"a":** I force-push the rewritten history. You then delete your local folder and re-clone, and run step 3 (measurements) in the fresh clone.
- **"b" or "c":** `git pull`, then run step 3 now.

Either way I'll catch your T12 push automatically and carry on with the README check, the Gemini triage and final acceptance.

```
measure.sh: building the program and the measurement tool
measure.sh: tests with the race detector and coverage
measure.sh: CODEHEIM's original program, and ours on the same
images
measure.sh: the program on the bundled photos, sequential mode
measure.sh: the program on the bundled photos, pipeline mode
measure.sh: the program on the bundled photos, workers mode
measure.sh: preparing 32 images
measure.sh: stage times, sequential mode, 32 images
measure.sh: benchmarks on synthetic photos, 5 runs (several minutes)
measure.sh: benchmarks on the bundled photos, 5 runs
measure.sh: CPU and memory profiles, sequential mode
measure.sh: whole program, 5 runs each
measure.sh: done; now run: scripts/tables.sh > results/tables.md &&
scripts/readme.sh
readme.sh: README.md tables updated from results/tables.md
[main 9d5dc88] T12 redo: measurements on the Mac with the new
photos
 14 files changed, 306 insertions(+), 306 deletions(-)
To https://github.com/mioluigi/go-image-pipeline.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/mioluigi/go-
image-pipeline.git'
hint: Updates were rejected because the remote contains work that
you do not
hint: have locally. This is usually caused by another repository pushing
```

to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

Show more

Comparing which photos the measurements were taken on ▸

The push was rejected because GitHub has a commit your copy doesn't: my photo fix (`3dd3bbd`). That also means **your measurements ran on the earlier photo files:** the sideways portrait (`FE3994CD…JPG`) and the three with location data. The timings would come out almost the same (same pixel counts), but `results/` would name a photo the repo no longer has and couldn't be reproduced from it. So these results shouldn't be pushed, and step 3 needs one more run on the fixed photos. Sorry for the extra 10 minutes.

You still need to choose **a**, **b** or **c** for the old photos in the history, because that decides how you get the fixed photos:

If (a), rewrite the history (my recommendation): reply "a" and wait. I'll force-push the cleaned history and tell you when it's done (a couple of minutes). Then:

```bash
cd .. && rm -rf go-image-pipeline
git clone https://github.com/mioluigi/go-image-pipeline.git && cd go-image-pipeline
scripts/measure.sh && scripts/tables.sh > results/tables.md && scripts/readme.sh
git add results README.md && git commit -m "T12 redo: measurements on the Mac with th
```

If (b) or (c), leave the history: run this now.

```bash
git fetch && git reset --hard origin/main      # drops only your unpushed results com
ls images                                       # must show IMG_portrait.jpg
scripts/measure.sh && scripts/tables.sh > results/tables.md && scripts/readme.sh
git add results README.md && git commit -m "T12 redo: measurements on the Mac with th
```

`git reset --hard` throws away your local results commit and makes your copy match GitHub exactly. That's intended here, since those results are being replaced.

Checking the remote history for the redo.

A T12 redo commit reached GitHub. I'll check which photos it was actually measured on.

Ran 4 commands ▸

```
Cloning into 'go-image-pipeline'...
remote: Enumerating objects: 246, done.
remote: Counting objects: 100% (246/246), done.
remote: Compressing objects: 100% (145/145), done.
remote: Total 246 (delta 113), reused 226 (delta 93), pack-reused 0 (from 0)
Receiving objects: 100% (246/246), 26.46 MiB | 39.73 MiB/s, done.
Resolving deltas: 100% (113/113), done.
measure.sh: building the program and the measurement tool
measure.sh: tests with the race detector and coverage
measure.sh: CODEHEIM's original program, and ours on the same images
measure.sh: the program on the bundled photos, sequential mode
measure.sh: the program on the bundled photos, pipeline mode
measure.sh: the program on the bundled photos, workers mode
measure.sh: preparing 32 images
measure.sh: stage times, sequential mode, 32 images
measure.sh: benchmarks on synthetic photos, 5 runs (several minutes)
measure.sh: benchmarks on the bundled photos, 5 runs
measure.sh: CPU and memory profiles, sequential mode
measure.sh: whole program, 5 runs each
measure.sh: done; now run: scripts/tables.sh > results/tables.md &&
scripts/readme.sh
readme.sh: README.md tables updated from results/tables.md
[main dd7fe42] T12 redo: measurements on the Mac with the new photos
 14 files changed, 306 insertions(+), 306 deletions(-)
Enumerating objects: 33, done.
Counting objects: 100% (33/33), done.
Delta compression using up to 10 threads
Compressing objects: 100% (17/17), done.
Writing objects: 100% (17/17), 7.52 KiB | 7.52 MiB/s, done.
Total 17 (delta 12), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (12/12), completed with 10 local objects.
To https://github.com/mioluigi/go-image-pipeline.git
  3dd3bbd..dd7fe42  main -> main
```

Show more

Working out whether he picked the clone path ▸

The redo worked: you re-cloned, measured on the fixed photos, and pushed cleanly (`dd7fe42`). I've checked it against the README:

- **Tables match** `results/` exactly, and every README sentence holds against the new numbers. I corrected one: robustness cost "no measurable time" is now "a few percent" (126 ms for the original versus 131 ms for ours). That fix is pushed.
- **Headline on your M1 Max (32 photos):** the worker pool is ×3.54 faster than sequential and the pipeline ×1.50. The pipeline's ceiling is ×1.54 because loading takes 65% of the time, so it got about as close to that ceiling as the theory allows.

- **Final acceptance check from a fresh clone:** 16 of 18 pass. The two left are the AI dialog PDFs and the last task ticks, which close as you finish below.

About the photo history: your re-clone used the steps from option (a), but I haven't rewritten anything, since you didn't reply "a". So the earlier photos, with location data, are still in old commits. If you want them removed, reply "a" and I'll do it before you export anything. Otherwise I'll leave the history as it is.

Remaining steps, in this order:

1. **T14: build and open the app.**

```bash
scripts/build.sh && open dist/ImagePipeline.app
```

Tell me it ran all three modes, and that `~/Desktop/ImagePipeline-output/IMG_portrait.jpg` looks upright and grayscale.
2. **T17: Gemini review.** Paste `gemini-repo-review-002-prompt.md` into Gemini with the repo URL, and send me its findings. I'll triage them into `docs/reviews/002-repository-review.md` and fix what's accepted.
3. **T16: AI dialogs.** After that, export this conversation and the Gemini repository review as PDFs, named `docs/ai-dialogs/claude-build-dialog.pdf` and `docs/ai-dialogs/gemini-repository-review.pdf`. Commit and push them.
4. **T18:** I run the final acceptance check, tick the last tasks, and you submit `https://github.com/mioluigi/go-image-pipeline.git` in Canvas.

a - remove the earlier photos to the one I updated the local folder

Settling which photos to remove from history ▸
Understood: **(a), remove the earlier photos from the history, keeping only the current set.** That's a little stronger than my dry run, which kept the old photos and stripped their metadata. So this rewrite does both:

- **Removes** the earlier photo files (`IMG_2462`, `IMG_2553`, `IMG_2658`, `IMG_2659`, and the sideways `FE3994CD…JPG`) from every commit.
- **Strips the metadata** from the one commit where your current three photos first appeared with GPS.
- **Leaves CODEHEIM's commits and photos untouched.**

Ran 4 commands ▸
The rewritten history is on GitHub (`2b97283`). Next, I'll bring my own copy onto it, record the decision in the log, and confirm with a brand-new clone that nothing old remains:

Sync my copy, record the rewrite, verify a fresh clone ▸

Done. The earlier photos are gone from the repository's history:

- **Removed from every commit:** the first photo set (`IMG_2462`, `IMG_2553`, `IMG_2658`, `IMG_2659`), the sideways `FE3994CD…JPG`, and the stray `.DS_Store`. The first versions of your current three photos keep their pixels but lost their location data.
- **Checked on a brand-new clone from GitHub:** of the 12 photo versions anywhere in history, none has location data, and your current files are unchanged.
- **CODEHEIM's three commits are untouched,** with the same IDs and GitHub signatures, so `run-original.sh` still works.
- **One commit was dropped:** "T02: four of my own photos" only added the first set, so it became empty. The rewrite is recorded in `docs/decisions.md`.

The one caveat: GitHub may keep the old, now-unlinked commits reachable by exact commit ID for a while, until its cleanup runs. Nothing on the repo page or in any new clone leads to them.

Important for your Mac: your current local folder still has the *old* history. Don't `git pull` or push from it, because a pull would merge the old photos back in. Replace it:

```bash
cd .. && rm -rf go-image-pipeline
git clone https://github.com/mioluigi/go-image-pipeline.git && cd go-image-pipeline
```

Then continue from that fresh clone:

1. **T14:** run `scripts/build.sh && open dist/ImagePipeline.app`. Confirm all three modes ran and that `~/Desktop/ImagePipeline-output/IMG_portrait.jpg` is upright.
2. **T17:** paste `gemini-repo-review-002-prompt.md` into Gemini with the repo URL, and send me its findings.
3. **T16:** export the two PDFs, `claude-build-dialog.pdf` and `gemini-repository-review.pdf`, into `docs/ai-dialogs/`. Commit and push them.
4. **T18:** I run the final acceptance check (16 of 18 already pass), and you submit `https://github.com/mioluigi/go-image-pipeline.git` in Canvas.

List the commands needed for what should be next

Run these in Terminal, one block at a time.

1. Replace your local folder with the cleaned history

```bash
```

```
cd ~ && rm -rf go-image-pipeline        # adjust the path if your clone lives elsewhe
git clone https://github.com/mioluigi/go-image-pipeline.git
cd go-image-pipeline
```

2. T14: build the executables and open the app

```bash
scripts/build.sh
open dist/ImagePipeline.app
open ~/Desktop/ImagePipeline-output/IMG_portrait.jpg
```

Check that the Terminal window shows all three modes, and that the photo is grayscale, upright and not stretched.

3. T17: Gemini repository review (in the browser, no command)

- Open Gemini and paste the contents of `gemini-repo-review-002-prompt.md`; it includes the repo URL.
- Paste Gemini's findings back here. I'll triage and fix them and push.
- Then pull my fixes:

```bash
git pull
```

4. T16: AI dialogs, after I've finished the triage

- In Claude: export this conversation as a PDF and save it as `docs/ai-dialogs/claude-build-dialog.pdf` in the repo folder. (If there's no export option, use your browser's print → Save as PDF.)
- In Gemini: save the repository-review conversation the same way, as `docs/ai-dialogs/gemini-repository-review.pdf`.

```bash
ls docs/ai-dialogs/            # gemini-spec-review.pdf, claude-build-dialog.pdf, gem
git add docs/ai-dialogs && git commit -m "T16: AI dialogs (Claude build conversation,
```

5. T18: final acceptance and submission

- Tell me "dialogs pushed". I'll run the final acceptance check from a fresh clone, tick the remaining tasks, and push.
- Submit this URL in Canvas: `https://github.com/mioluigi/go-image-pipeline.git`

I am confused with step T17 - break it down such that I can follow it

Including the full prompt text for copying ▸

T17 just means asking Gemini to grade the finished repository, then bringing its answer back to me. Five steps:

Step 1: Open a new Gemini chat
Go to **gemini.google.com** in your browser and click **New chat.**

Step 2: Copy this whole block and paste it into Gemini, then press Enter

```
You are grading a finished graduate Go assignment (Northwestern MSDS 431, Week 6:
Data Pipelines with
Concurrency). Act as a strict grader applying a five-criterion rubric (20 points
each: coding rules and
organization; testing and software metrics; design and development; documentation;
application) and as a
senior Go engineer reviewing concurrent code for production.

Repository: https://github.com/mioluigi/go-image-pipeline

The project replicates CODEHEIM's goroutine image pipeline, adds error handling,
tests, benchmarks, a
sequential mode and a worker-pool mode, and compares processing times. Its spec
(specs/001-concurrent-image-pipeline/spec.md) lists acceptance criteria AC-1 to AC-
11. Do not write code.

Report:
1. Rubric score. For each criterion, the rating you would give (20 / 15 / 10 / 5 /
0) and the specific
   evidence or gap that decides it.
2. Assignment steps. Any of the ten assignment steps (docs/assignment-and-
rubric.md) not visibly satisfied,
   including the GenAI Tools section and conversation logs.
3. Concurrency review. Read pipeline/modes.go and pipeline/pipeline.go: any
deadlock, goroutine leak, race,
   or ordering bug you can construct an input for; whether the timing comparison
between modes is fair.
4. Evidence check. Do the README's conclusions and recommendation follow from
results/tables.md? Flag any
   claim that over-reaches the data (one machine, four photos).
5. Grader friction. Anything that would stop a grader cloning, building, testing
and running the program
   from the README alone.

Format: numbered findings, each with severity (High / Medium / Low), where (file
and section), the problem,
and a suggested change in one or two sentences. End with the total score out of 100
and a verdict: submit
as is, fix then submit, or needs rework. Today is the due date: prefer findings
that can be fixed in under
an hour.
```

Step 3: Only if Gemini says it can't open the GitHub link

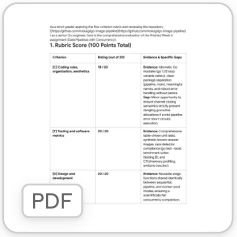
1. Run this in Terminal to open your repo folder in Finder:

```bash
open ~/go-image-pipeline
```

2. In Gemini, click + (Upload files) and attach these 7 files from that folder:
   - `README.md`
   - `results/tables.md`
   - `main.go`
   - `pipeline/modes.go`
   - `pipeline/pipeline.go`
   - `specs/001-concurrent-image-pipeline/spec.md`
   - `docs/assignment-and-rubric.md`
3. Type "Please review using these files" and press Enter.

Step 4: Bring the answer back
When Gemini finishes, copy its **entire reply** and paste it here. I'll decide each finding (accept, adjust or reject), fix what's accepted, and push.

Step 5: Keep that Gemini chat open
You'll save it as a PDF in T16 (`docs/ai-dialogs/gemini-repository-review.pdf`), so don't close or delete it.



Created a file, ran 3 commands ▸

T17 is done: Gemini gave the repository **98/100, "submit as is"**.

- **The one finding is rejected, with evidence.** Gemini suggested adding cancellation in case a failed photo leaves goroutines stuck. The design already prevents that: a failure travels with its photo, and no stage stops reading early. The leak test passed 50 runs today under the race detector, with failing photos in every mode, so the change would add code with nothing to test.
- **Two statements in the review were wrong about the repo.** Gemini said the README blames a ×2 resize bottleneck (it says loading, with a ceiling of ×1.54), and that all the AI dialogs are already committed. I recorded both in `docs/reviews/002-repository-review.md`. They suggest Gemini didn't read the current results closely, so its score should carry less weight.

- **No code changes needed.** I saved Gemini's review PDF as `docs/ai-dialogs/gemini-repository-review.pdf` for you, and it's all pushed.

What's left, in order:

1. T14: tell me whether the `.app` ran. If you haven't run it yet, from your fresh clone:

```bash
git pull
scripts/build.sh && open dist/ImagePipeline.app
open ~/Desktop/ImagePipeline-output/IMG_portrait.jpg
```

Reply "app OK" if the window showed all three modes and the photo is grayscale and upright.

2. T16: the Claude conversation as a PDF. It's the only dialog file still missing.

- Export this conversation to PDF. If there's no export button, use your browser's Print and choose Save as PDF.
- Save it into your repo folder as `docs/ai-dialogs/claude-build-dialog.pdf`, then:

```bash
git pull
git add docs/ai-dialogs/claude-build-dialog.pdf && git commit -m "T16: Claude build c
```

3. T18: reply "dialogs pushed". I'll run the final acceptance check from a fresh clone and tick the last tasks, then you submit `https://github.com/mioluigi/go-image-pipeline.git` in Canvas.

app OK

Started a watch 2m 24s ▸

T14 is ticked and pushed. Only T16 (your part) and T18 remain.

T16: the Claude conversation as a PDF

1. Export this conversation to PDF. If the app has no export option, open the conversation in your browser and use **File → Print → Save as PDF**.
2. Save it into your repo folder as `docs/ai-dialogs/claude-build-dialog.pdf`.
3. Then, in Terminal:

```bash
```

```
cd ~/go-image-pipeline
git pull
git add docs/ai-dialogs/claude-build-dialog.pdf && git commit -m "T16: Claude build c
```

I'm watching GitHub for that file. When it arrives I'll do T18 myself: add a short index of the dialog files, tick T16, T17 and T18, and run the final acceptance check from a fresh clone. Then I'll tell you it's ready to submit: `https://github.com/mioluigi/go-image-pipeline.git` in Canvas.